exaStamp Practical Course

exaNBody Summer School 2025

Introduction

The main goal of the present tutorial is to help you get acquainted with the `exaStamp` code. This document aims at guiding you setting up a complex classical molecular dynamics simulations involving different key functionalities of both `exaStamp` and `exaNBody` codes. The tutorial contains one mandatory exercise (**Exercise 1** folder) divided into 3 different parts each associated with a separate subfolder:

- **Exercise1/Part1_SampleSetup :** Setup the sample by defining the domain, the particles type as well as thermostat,
- **Exercise1/Part2_DirectShock :** Setup a direct sustained shock simulation by applying a constant velocity to a piston over time,
- **Exercise1/Part1_ReverseBallistic :** Setup a reverse ballistic shock simulation by freezing the piston and applying a velocity field to the remaining part of the sample.

An additional exercise will be offered to the attendees depending on the remaining time. You will have the choice between creating a dynamic impact between two spherical particles or manipulate the NPT algorithm that allows one to drive a microscopic sample to a specific thermodynamic state or through a specific thermodynamic path.

For the sake of keeping simulation (computer-wise) time reasonable, the considered systems are quite small but you are free to modify the systems' size at your own risk, and depending on the available local ressources. In each

subfolder, a **.msp** solution file combining the answers to the previous questions can be found in the **dir.solutions** folder.

Reminder: searching and debugging is part of the learning process. Use **exaStamp –help operator_name** whenever you want. However, as **exaStamp** is a constantly evolving code undergoing a strong effort in terms of documentation and error messages, some outputs might not be explicit enough yet. Do not hesitate to write down comments, suggest improvements or reach out to the organizers if you have any question.

Exercise 1: Performing a shock simulation

Part 1: Sample Setup

Use the **Part1_baseline.msp** file and follow the questions below to complete it.

Question 1: Setup the EAM alloy formalism and atomic species

Using the **includes** YAML block, please add the configuration file for EAM located in the **dir.configs** folder. In addition, use the **species:** to declare the specy **"Ta"** through a dictionary with an atomic mass equal to 180.95 Da, an atomic number set to 73 and a partial charge equal to 0.

Remark:

At this time, please change the **types** list of the **lattice** operator of the **setup_system** block so the lattice generator can map the required atom type to the one listed using the **species** block.

Solution : Expected screen output

```
======== Atom species ========
- Ta: { mass: 1.80950000e+02 , z: 73 , charge: 0.00000000e+00 }
==============================
```

Question 2: Domain definition

Define the domain with a cell size of 6.6 Å on a 50x5x5 grid and an XForm matrix equal to identity. Make sure the domain bounds are consistent with the cell size and the grid dimensions.

In addition, unset the periodic boundary along the X-axis and set the expandable boolean to true such that the domain automatically expands when particles reach its boundary.

Solution : Expected screen output

```
======= Simulation Domain =======
bounds    = (0,0,0)-(330,33,33)
dom size  = 330,33,33
grid      = 50,5,5
cell size = 6.6
grid size = 330,33,33
periodic  = false , true , true
mirror    =
xform     = {(1,0,0),(0,1,0),(0,0,1)} (identity)
inv_xform = {(1,0,0),(0,1,0),(0,0,1)}
scale     = 1 / 1
=================================
```

Question 3: Particle Regions

In the **setup_system** block, use the **particle_regions** operator to define two regions: 1) a region named **BAR** of type **bounds** spanning $X \in (50, 300)$ Å and transverse bounds covering the domain dimensions; 2) a region named **PISTON** of type **bounds** with $X \in (10, 50)$ Å and transverse bounds covering the domain dimensions.

Usage example:

```
particle_regions:
  - REGION1:
      bounds: [ [ 0 ang, 0 ang, 0 ang], [ 100 ang, 100 ang, 100 ang] ]
```

Question 4: Particles creation

You might have noticed that the lattice operator in the **setup_system** block defines a simple cubic lattice (SC) which is not the appropriate one for Tantalum. Modify the lattice operator using the body-centered cubic (BCC) structure with a lattice parameter set to 3.3 Å in each direction. Note that BCC contains two atoms per unit cell so please adapt the types list accordingly. Finally, specify that particles should be created on a spefici spatial region (using the **region** property) that is the union between **BAR** and **PISTON** regions defined above.

Solution : Expected screen output

```
======= Lattice generator =======
structure         = BCC
types             = Ta Ta
lattice cell size = 3.3,3.3,3.3
position shift    = 0,0,0
noise sigma       = 0
noise cutoff      = -1
lattice start     = -1 , -1 , -1
lattice end       = 101 , 11 , 11
output particles  = 17500 (min=17500,max=17500)
================================
```

Question 5: Define a tracked group of particles in a region

Finally, still in the **setup_system** block, use the **track_region_particles** to define the particles located in the **PISTON** region as tracked under the name "PistonGroup".

Usage example:

```
track_region_particles:
  expr: MYREGION # boolean expression combining previously defined regions
  name: MYNAME   # name of the tracked ensemble of particles
```

Question 6: Force computation setup with an NVT to NVE transition

1) At the root of the **.msp** file, define a **langevin_thermostat** operator with a target temperature **T** set to 300 K and a friction coefficient γ set to 0.1 ps^{-1}.

2) Define a batch operator named **trigger_nvt_to_nve** that uses the **greater_than** operator on the **physical_time**. Using the **rebind** operator, the input **value** should be replaced by **physical_time** while the output **result** should be put in the boolean **is_nve**. The boolean **is_nve** should be set to true if the physical time is greater than 0.25 ps.

Usage example:

```
my_operator:
  rebind: { value: my_input, result: my_output}
  body: [ greater_than: { threshold: 10. fs }]
```

3) Define two operators **integrate_nvt** and **integrate_nve** that contain a **body** activated if **is_nve** is false and true respectively. In the NVT case, both the **langevin_thermostat** and **eam_alloy** operators should be used while the NVE case should only call the **eam_alloy** operator.

Usage example:

```
my_conditional_operator:
  condition: my_boolean
  body: [ operation1, operation2 ]
```

Finally, add the operators defined above as a list to the **compute_force** operator.

Question 7: Dump functions and print message

1) Define a batch operator named **trigger_nvt_to_nve_msg** that uses the **nth_timestep** operator. Using the **rebind** function, the input **value** should be mapped to **timestep** while the output **result** should be send to the boolean **nvt_to_nve_step**. The boolean **nvt_to_nve_step** should be set to true if the timestep is equal to 50, corresponding to a physical time of 0.25 ps.

Usage example:

```
my_operator:
  rebind: { value: my_input, result: my_output}
  body: [ nth_timestep: { at_timestep: 1000 }]
```

2) Define a conditional operator named **print_nvt_to_nve_msg** that uses the **message** operator only if the boolean **nvt_to_nve_step** is true.

Usage example:

```
my_operator:
  condition: my_condition
  body: [ message: { msg: 'my message', endl: true }]
```

Finally, add the operators defined above as a list to the **compute_force** operator. In addition, change the simulation max iteration to 100 in the **global** block and check if the message is correctly printed to your screen.

3) Complete the **write_snapshot** block so it uses the **write_xyz** or **write_paraview** operators. Prior to calling these operators, you should also add the **timestep_file** operator that allows to define the file name pattern using the current timestep. Include the **dir.outputs** folder in the file pattern. In addition, set the dump data frequency to 10 in the **global** block.

Usage example:

```
timestep_file: 'my_xyz_pattern_%09d.xyz'
write_xyz: { fields: [ 'myfield1', 'myfield2' ] }
timestep_file: 'my_pv_pattern_%09d'
write_paraview: { fields: [ 'myfield1', 'myfield2' ] }
```

You should be able to visualize the trajectory using either Ovito or ParaView. A this time, the particles should only vibrate due to thermal fluctuations and no rigid body movement should be observed.

Part 2: Direct Shock Simulation

Switch to the folder **Part2_DirectShock** and either 1) copy your working **.msp** file from Question 7 or use the **Part2_baseline.msp** file.

The simulation as designed previously is set to run for 100 steps with the first 50 steps combining a Langevin thermostat with the EAM potential while the last 50 steps do not involve thermostatting. You will now impose a constant velocity to the piston part of the system from timestep 50 onward, i.e. to impose a direct sustained shock to the sample.

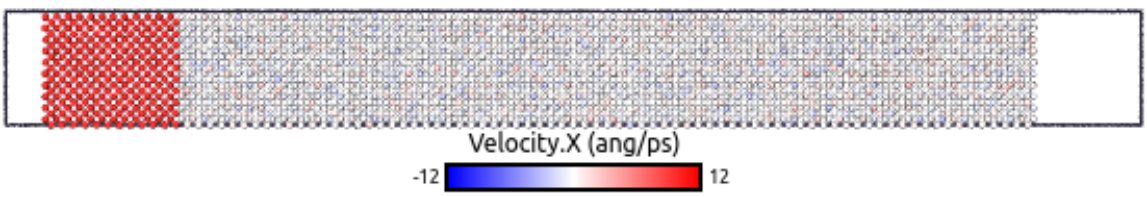

Figure 1: X-component of the atoms velocity field at iteration 50 for a direct shock setup.

Question 8: Impose velocities and forces to the piston

1) Define an operator **piston_velocity** that combines both **set_velocity** and **set_force**. A velocity of 1000 m/s along the x-axis should be imposed while the forces on atoms should be set to 0. You should precise that these operations are to be performed only to the tracked region previously labelled as **PistonGroup**.

Usage example:

```
set_velocity: { value: [ 1, 2, 3], region: YourRegion }
set_force: { value: [ 10, 10, 10] }
```

2) Add the operator **piston_velocity** as defined above to the operator that performs the integration in the NVE ensemble. This way, the piston should move at the time the Langevin thermostat is deactivated. You can run the case again and visualize if something happens at timestep 50, which should be the case.

You can modify the simulation end timestep so it can run a bit longer (1000 steps for example). In addition, change the dump frequency to 50 and the log frequency to 10. If you visualize the X-component of the velocity, you should obtain something similar to Figure 1.

Letting the system evolve in time will generate the propagation of an elastic wave with no effect but compression through the sample, followed by a plastic wave inducing defects in the sample. You can play around with the imposed velocity to observe its influence, before jumping to the second shock scenario of this tutorial.

Part 3: Reverse Ballistic Setup

Switch to the folder **Part3_ReverseBallistic** and either 1) copy your working **.msp** file from Question 7 or use the **Part3_baseline.msp** file.

In this part, you will now assign a constant negative velocity to the bar at time step 50 only while freezing the piston from timestep 50 onward, i.e. to impose a reverse ballistic shock to the sample.

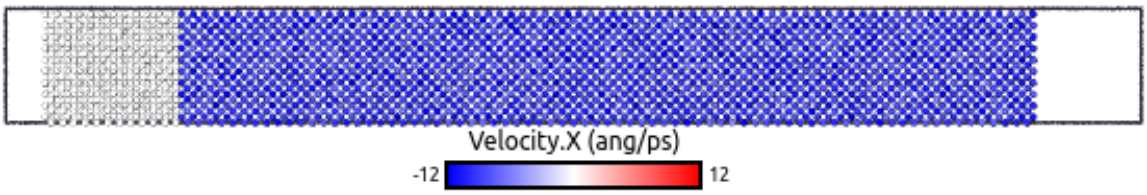

Figure 2: X-component of the atoms velocity field at iteration 50 for a reverse ballistic shock setup.

Question 9: Freeze the piston

1) Define an operator **freeze_piston** that combines both **set_velocity** and **set_force**. This time, both velocities and forces should be set to 0. You should precise that these operations are to be performed only to the tracked region previously labelled as **PistonGroup**.

Usage example:

```
set_velocity: { value: [ 1, 2, 3], region: YourRegion }
set_force: { value: [ 10, 10, 10] }
```

2) Add the operator **freeze_piston** as defined above to the operator that performs the integration in the NVE ensemble. This way, the piston should be frozen starting at the time at which the Langevin thermostat is deactivated.

Question 10: Initialize sample's velocity

You previously defined an operator that prints a message at timestep 50 informing the user that the Langevin thermostat is deactivated and that you're switching to the NVE ensemble. Add a **shift_v** operation to this operator such that the entire system (but the piston) receives an additional negative velocity of 1000 m/s along the X-axis at this time step only.

Usage example:

```
shift_v: { value: [ 1, 2, 3], region: YourRegion }
shift_v: { value: [ 0, 2, 3] }
```

You can run the case again and visualize if something happens at timestep 50. You can modify the simulation end timestep so it can run a bit longer (1000 steps for example). In addition, change the dump frequency to 50 and the log frequency to 10. If you visualize the X-component of the velocity, you should obtain something similar to Figure 2.

Letting the system evolve in time will also generate the propagation of an elastic wave followed by a plastic wave depending of the impact velocity. You can play around with the simulation parameters and observe the differences between this reverse ballistic shock setup and the direct shock one.

Exercise 2: Impact between two nano particles

Question 1: Domain

Create a 3D-periodic domain centered around (0,0,0) with dimensions 400x400x400 Å over a grid with a cell size set to 10 Å.

Question 2: Regions

Create two separate ellipsoidal regions as a list with the **particle_regions** operator using a quadric of shape sphere.

Usage example:

```
particle_regions:
  - MYSPHERE:
      quadric:
        shape: sphere
        transform:
          - scale: [ 1 ang , 1 ang , 1 ang ]
          - translate: [ 0 ang , 0 ang , 0 ang ]
```

Question 3: Particles

Populate the two regions using the lattice operator already present in the baseline file.

Question 4: Dynamics

Inspired by the first exercise, try to imagine a scenario involving:

- A Langevin thermostat over 0.25 ps,
- A switch to NVE ensemble (i.e. deactivating the Langevin thermostat) after that time,
- An initialization of nano particles velocities at the exact same time without but not after. The assigned velocities should result in the impact between the two nano particles,
- Dump xyz or paraview files with the field processor_id in order to check if the load balancing is actually working.

knowing that the force computation from the EAM potential can be called using the **eam_alloy** operator.

You can then run the case using **OMP_NUM_THREADS=1 mpirun -np 4 Exercise2_baseline.msp** (if applicable) and visualize with Ovito or ParaView the field **processor_id**